Software Process

The Struggle to Manage Complexity: Waterfall to Agile Process

The Invention of Software Process

- The Problem: Massive Project Failures in the 1960s
- The Cause of the Problem: Software systems became increasingly **large**, **complex**, and difficult to **manage**.

Software engineers began to recognize the growing challenges of managing software complexity.

- Problems included missed deadlines, cost overruns, unreliable systems, and maintenance nightmares.
- This led to the first NATO Software Engineering Conference (1968).

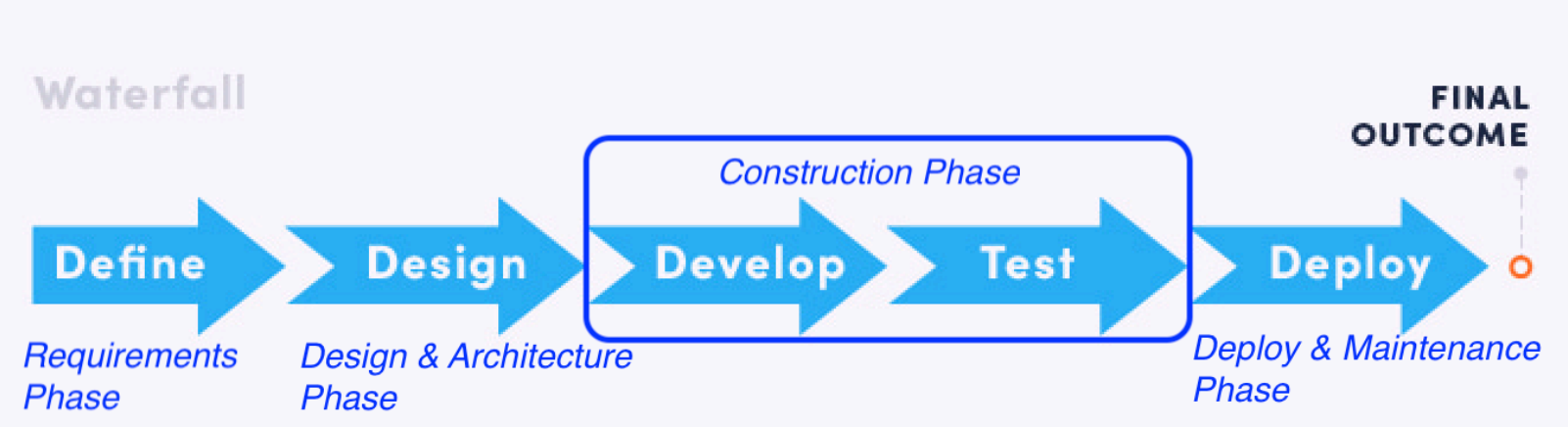


Waterfall Process Model

- The Proposed Solution: A sequential, phase-based model
- The Inspiration: Hardware Engineering
- By: Winston W. Royce, who described it in his 1970 paper titled "Managing the Development of Large Software Systems." (The name "Waterfall" was popularized later.)

Manufacturing had clear phases (design → prototype → production), why not software?

- The Flow: In its simplest form, we (1) define requirements, (2) design, (3) implement, and (4) test software, then maintain it.



SE Rules Applied - Process

- Divide and Conquer
- SRP (Single Responsibility Principle)
- Only Fools Rush In: We don't construct unless we know what to construct: Requirements and Architecture/Design are required to know what to construct.

What else?

The Attraction (Success) of the Waterfall Model

The Waterfall model appeared attractive because it promised **structure and control** through:

- Clear phases (Requirements → Design → Implementation → Testing → Maintenance)
- Detailed documentation at each stage
- Management checkpoints and approvals
- Predictable timelines and budgets

The Waterfall approach was **widely used** from the 1970s through the early 2000s, but it was risky when requirements were uncertain or changing.

- It still works well `(only) when requirements are stable`, or when regulation requires documentation (e.g., military or safety-critical systems).
- Its strict, single-pass sequence struggles with `late feedback`, which helped drive the rise of iterative and Agile methods.

Why Did a Strict Single-Pass Waterfall Struggle with Real-World Complexity?

1. Requirements Change

Linear Thinking ≠ Complex Reality

```
Waterfall Assumption:          Reality:
A → B → C → D                  A ↔ B
                               ↕   ↕
                               D ↔ C
```

By the time you finish a 2-year waterfall project, the world and its requirements may have changed!

2. "Big Bang" Integration

- Modules developed separately for months
- Integrated only at the end
- Result: **Integration Hell** - components don't work together

> Example: Team A's login module expects JSON, Team B's database returns XML

3. Late Testing = Expensive Rework

- If you don't test until the end, problems are found when they are most expensive to fix.
- The later a defect is found, the more design, code, tests, and documents depend on it.

```
Found in:      Requirements   Design   Coding   Testing   Deployed
Relative cost:       1x           ↑        ↑↑       ↑↑↑       ↑↑↑↑
```

4. Little Feedback

```
Strict single pass: Requirements → [BLACK BOX] → Final Product
```

- Complexity requires **continuous learning**
- A strict single pass provides **few mechanisms to learn and adapt**

IMPORTANT

The Fundamental Risk of Strict Waterfall (The Confusion of Complicated vs. Complex)

- Strict Waterfall treats software like a **complicated** problem (like building a bridge) when it is often a **complex** problem (like managing an ecosystem).
- Complex problems need iterative exploration, not only upfront planning.

Software engineers learned it the hard way.

Complicated vs. Complex

The complicated: many parts, but knowable

- It can be hard because of the number of components.
- Experts can analyze it and break it into manageable parts.

The complex: unpredictable, with interacting parts

- Interdependencies and emergent behavior make cause and effect hard to predict.
- "Unknown unknowns" often appear only in hindsight.
- It needs adaptive, iterative approaches, not rigid, linear ones.

A Real Example: The FBI's Virtual Case File (2000–2005)

- Cost: about $170 million
- Result: Abandoned in 2005
- The delivered system was judged inadequate late in the project, after years of work.

Many people cite this as a Waterfall failure. **Is that the whole story?**

What Did the Investigators Find?

The Department of Justice Inspector General's report pointed to several causes:

- Poorly defined and **continually changing requirements**
- Weak IT investment management and contract oversight
- Turnover in leadership and lack of continuity
- An overly ambitious schedule

Notice: Waterfall assumes **stable** requirements, yet requirements kept changing here.

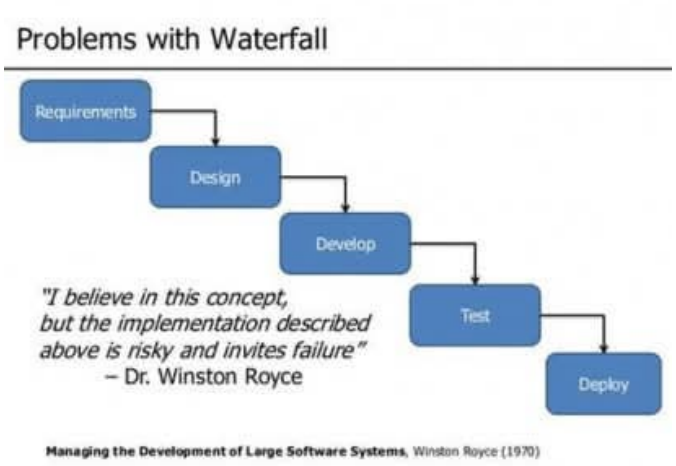
Unknown unknowns, changes, and interdependencies (coupling) make the "stable requirements" assumption unrealistic.

The Irony of the Waterfall Model

In his paper, Royce presented this sequential approach (requirements → design → implementation → testing → maintenance) and then argued that this simple implementation was **risky** and prone to failure.

- Royce was actually critical of the simple version of it.





- Then, he spent most of the paper explaining the risks and proposing five additions:
  - Do the program design first
  - Document the design thoroughly
  - "Do it twice": build a pilot version first
  - Plan, control, and monitor testing
  - Involve the customer

Note: Royce favored **more** documentation, not less. His point was that `a single pass without feedback` is `dangerous`.

> ### The Lessons Learned from the Waterfall Model
>
> > Structure alone doesn't manage **complexity**—you need **adaptive** processes with fast feedback loops!

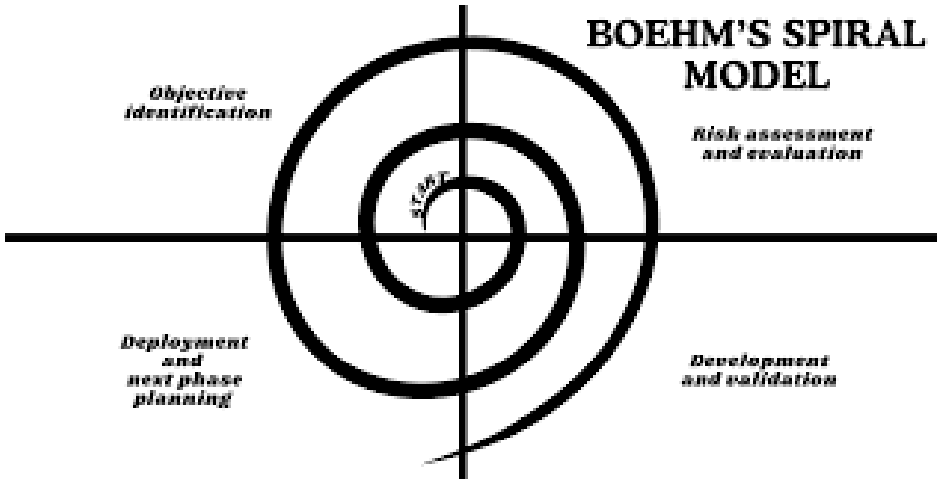
Now, we understand the goal is to `manage complexity`, not `complication`, but how?

The Spiral Model (1986–1988)

Barry Boehm proposed the **Spiral Model** (first published in 1986, and widely cited through his 1988 IEEE Computer paper) as an influential, risk-driven alternative to the Waterfall approach.

- The Spiral Model emphasized (1) **iterative development** and (2) **continuous risk evaluation** at every stage.





- He managed risks continuously throughout each cycle.
- His model blended Waterfall's structure with iterative flexibility.

Versioning Reflects Incremental Releases

The version naming convention (e.g., DOS 3.1 → 3.2, Windows 10 → 11) is a familiar example of iterative, incremental evolution, which is also central to Barry Boehm's **Spiral Model** principles:

- Each version marks a new iteration.
- Teams can evaluate risks and improvements each cycle.
- Software evolves continuously, not in a single release.

(Version numbers alone don't prove that a team does risk analysis; they only show incremental releases.)

Spiral Improved Risk Management, but Gaps Remained

What Spiral Contributed:

✅ Risk management

✅ Iterative development

✅ Early prototyping

Where Spiral-Style Projects Often Struggled:

❌ Often applied with heavy documentation
❌ Often required extensive planning for each cycle
❌ Cycles were often measured in months
❌ Focused more on process and risk than on day-to-day collaboration

The 1990s: Growing Frustration

Software Industry Problems:

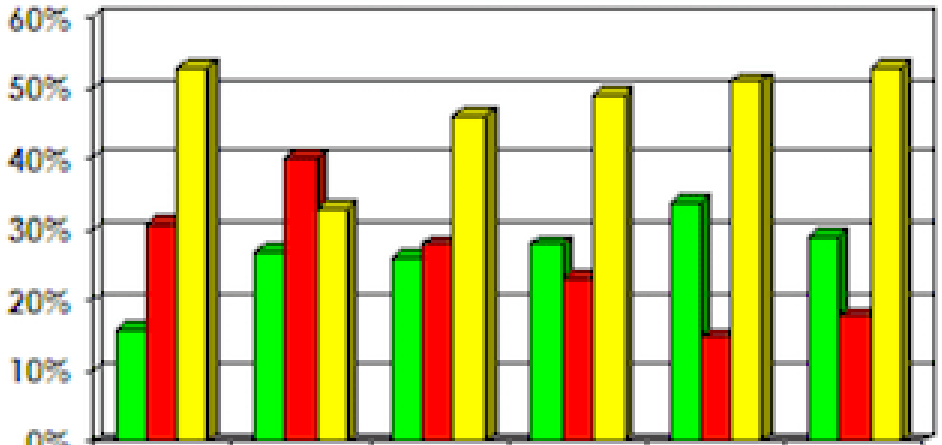
Standish Group CHAOS report (1994)

- About 16% of projects succeeded
- About 53% were "challenged" (late, over budget, or reduced scope)
- About 31% were cancelled
- Widely cited, but its methodology is debated

THE STANDISH GROUP

CHAOS Research

Studies



	1994	1996	1998	2000	2002	2004
Succeeded	16%	27%	26%	28%	34%	29%
Failed	31%	40%	28%	23%	15%	18%
Challenged	53%	33%	46%	49%	51%	53%

The Path to Agile: Key Milestones

Shifting the Emphasis: People and Collaboration

Boehm focused on **RISK**. Later methods added a strong emphasis on **PEOPLE and COLLABORATION**.

```
Spiral Model:
  "How do we reduce technical risk?"

Later iterative methods:
  "How do we empower people to build
   what customers actually need, and learn quickly?"
```

Both reduce risk.

- The later methods do it mainly through short feedback loops with customers.
- Boehm himself later wrote about balancing agility and discipline.

Rapid Application Development (RAD) - 1991

- James Martin
- Focus: Build fast, iterate faster
- Introduced: Time-boxing (fixed deadlines)

Scrum - 1995

- Jeff Sutherland & Ken Schwaber
- Rugby metaphor: Self-organizing teams
- Introduced: Sprints, Daily standups

Extreme Programming (XP) - 1996

- Kent Beck
- Focus: Technical excellence
- Introduced: Pair programming, TDD, Continuous Integration

Feature-Driven Development - 1997

- Jeff De Luca with contributions from Peter Coad
- Focus: Short iterations delivering features

February 2001: The Agile Manifesto

17 Software Developers met in Snowbird, Utah

Including:

- Kent Beck (XP)
- Jeff Sutherland (Scrum)
- Martin Fowler (Refactoring)
- Robert C. Martin (Uncle Bob)
- And 13 others


Manifesto for Agile Software Development
We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:
Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan
That is, while there is value in the items on
the right, we value the items on the left more.

The Manifesto Is Broader Than "People Over Process"

Four values (we value the items on the left more):

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Principles also cover:

- Frequent delivery of working software
- Sustainable pace
- Continuous attention to technical excellence
- Regular reflection and adjustment (retrospectives)

Comparing the Models

Both are iterative. They differ mainly in **emphasis** (typical, not absolute):

Aspect	Spiral (typical)	Agile (typical)
Focus	Risk-driven decisions	Customer value, feedback
Cycle Length	Months	Weeks (1-4)
Documentation	Often more formal	Just enough
Team Structure	Often role-based	Cross-functional, self-organizing
Planning	Risk-driven, per cycle	Adaptive, rolling
Customer	Reviews at end of cycles	Ongoing collaboration

Key Shift in Emphasis (From Risk-Driven Planning to Collaborative Adaptation)

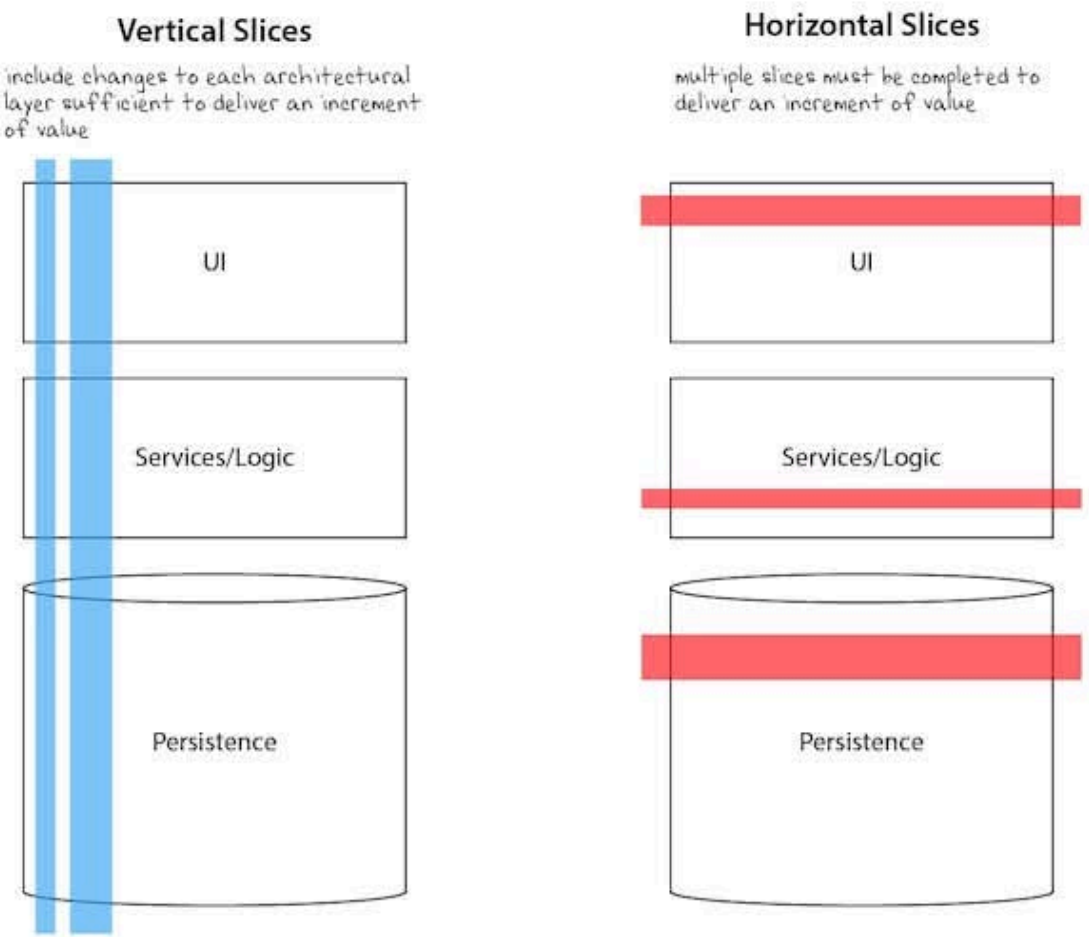
1. Spiral Model Mindset:

```
Plan → Analyze Risk → Build → Review → Replan
(Guiding question: "What could go wrong?")
```

2. Agile Mindset:

```
Collaborate → Build Small → Get Feedback → Adapt
(Guiding question: "What creates value now?")
```

Vertical Slice vs. Horizontal Slice



humanizingwork.com

Slicing Is a Way to Organize Work, Not a Process Model

- Horizontal and vertical slicing can be used inside **any** process model.
- A Spiral project can build vertical slices. An Agile team can (badly) build layer by layer.
- Agile practices **emphasize** vertical slices because they produce working, testable software in every iteration.

Example: Building an E-commerce Site

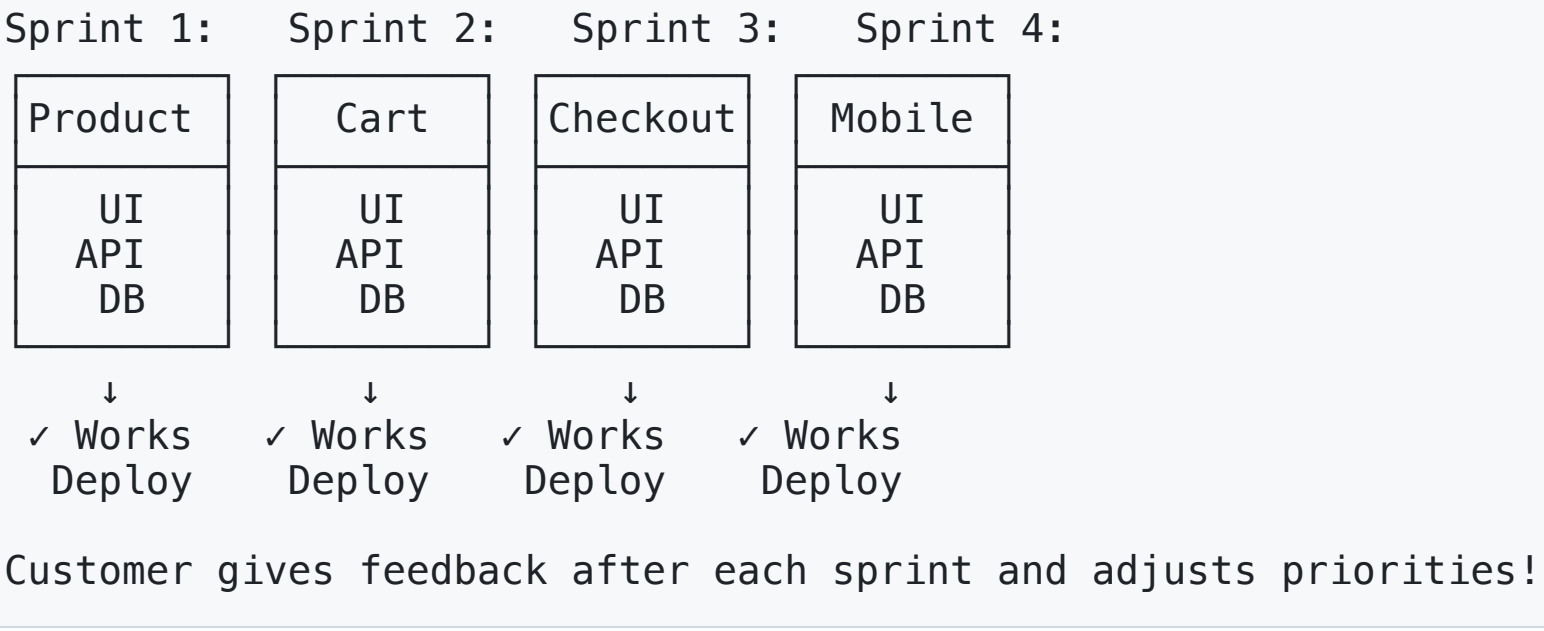
1. Layer-by-Layer (Horizontal Slice)

```
Month 1–2: Plan everything
Month 3–5: Build all layers horizontally

      ┌──────────────────────────────────────┐
      │ Products │ Cart │ Checkout │ Mobile │ ← UI Layer
      ├──────────────────────────────────────┤
      │ Products │ Cart │ Checkout │ Mobile │ ← API Layer
      ├──────────────────────────────────────┤
      │ Products │ Cart │ Checkout │ Mobile │ ← DB Layer
      └──────────────────────────────────────┘
                  ↓
Month 6: Integration + Customer sees it first time
  – Customer: "This isn't what we wanted!"
  – 6 more months to fix
```

❌ **Problem**: Nothing works end-to-end until month 6. Late feedback. High risk.

2. Feature-by-Feature (Vertical Slice)



✅ **Benefit**: Working feature every 2 weeks. Early feedback. Lower risk.

Key Differences (illustrative numbers):

	Layer-by-Layer (Horizontal)	Feature-by-Feature (Vertical)
Build	All features, one layer at a time	One feature, all layers at once
First working feature	Late (e.g., month 6)	Early (e.g., week 2)
Customer feedback	After months	Every iteration
Risk	High (big bang integration)	Lower (incremental)

Simple Analogy:

- **Horizontal**: Build foundation for entire house → frame entire house → finish entire house
    - *Can't use any room until everything is done*
- **Vertical**: Build complete kitchen → build complete bedroom → build complete bathroom
    - *Use the kitchen while building the bedroom!*

What Agile Added to Iterative Development

1. Speed to Market

- Spiral-style projects: Often months per iteration
- Agile: Working software every 1-4 weeks

2. Real Feedback

- Spiral-style projects: Prototypes and reviews
- Agile: Real users with real software

3. Adaptability

- Spiral-style projects: Change is handled at the next cycle's planning
- Agile: Change is welcomed every sprint

4. Team Empowerment

- Spiral-style projects: Often architects analyze risk, developers build
- Agile: Self-organizing teams make decisions

5. Less Waste

- Spiral-style projects: Often extensive documentation and analysis
- Agile: Just enough, just in time

6. Continuous Delivery

- Spiral-style projects: Big releases after each spiral
- Agile: Small releases continuously

The Technical Practices That Made Agile Possible

Test-Driven Development (TDD)

From the XP's Technical Innovations

1. Red → Green → Refactor

```
# RED: write the test first. It fails because ShoppingCart does not exist yet.
def test_shopping_cart():
    cart = ShoppingCart()
    cart.add_item("Book", 10.00)
    assert cart.total() == 10.00
```

2. Write the minimum code to pass the test

```python
# GREEN: the simplest implementation that makes the test pass.
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        self.items.append((name, price))

    def total(self):
        return sum(price for _, price in self.items)
```

Then **REFACTOR** with the test as a safety net.

Continuous Integration (CI)

- Code integrated multiple times per day
- Automated tests run on every commit
- Problems found within minutes, not months

Pair Programming

- Two developers, one computer
- Continuous code review
- Knowledge sharing

Refactoring

- Constantly improve code structure
- Enabled by automated tests

Conclusion: Match the Process to the Problem

The lesson is not "Waterfall failed, Agile won." It is to **design the feedback cycle and level of control to fit the problem**:

- **High uncertainty, fast-changing needs** → short iterations, frequent customer feedback
- **Stable, well-understood requirements** → more upfront planning can work well
- **Regulated or safety-critical systems** → documentation, traceability, and formal verification matter
- **Team size, skills, and contracts** → shape which practices are realistic
- Many real projects use a **hybrid** of plan-driven and Agile practices

Discussion: Which process would you choose for (1) a pacemaker's firmware, (2) a student-club mobile app, (3) a government payroll system? Why?